

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA152 Microcontrollers

Last Name: _____ Date: _____
First Name: _____ Subject/Section: _____

Laboratory Activity No. 3

Activity Recognition

100 Points

Overview

In this laboratory activity, you will build a full-stack Internet of Things system for real-time classification of user activities.

You will use the accelerometer and gyroscope sensors onboard the embedded platform/development board to send data to a cloud service, which will collect the data, classify it using machine learning, and allow a mobile app to read and display it.

This project involves a range of independent tasks across multiple platforms, including:

- Developing firmware for an embedded system
- Collecting readings for different activities
- Training and deploying appropriate machine learning algorithms for motion detection
- Developing a back-end web interface and associated database
- Mobile app development.

Since you are a 4th-year BS in Computer Applications, Embedded Systems Major, you should have most of these skills in place, in particular:

- Reading sensors
- Writing a mobile app

The new key elements that you'll need to explore are:

- Wi-Fi communication from the development board
 - Although you could also choose BLE again and use your smartphone app as a gateway
- Cloud service management to run virtual machines
- Machine learning to classify sensor data
- Web framework development to provide an API for the app

Requirements

In this lab, You will be using the following resources:

- An RT-Thread RT-Spark Development board with embedded gyroscope, accelerometer, and Wi-Fi transceiver
- An Android device for testing and debugging

- Appropriate USB cable (usually USB-C) for connecting the mobile device to the computer running Android Studio.
- Access to a cloud platform, such as Google, Microsoft or Amazon. Any of these providers will work, as you'll be using Infrastructure as a service to start a virtual machine.
- VSCode PlatformIO
- GitHub Account

You will also be using the following software:

- Android Studio for app development. This is the Android development environment, which includes the Android Software Development Kit (SDK), the toolchain, and an emulator.
 - This can be downloaded free of charge from the Android Developer Tools page: <https://developer.android.com/studio>
- To configure the development board, you'll need the STM32 CubeMX generator application, which you can download free of charge from here: <https://www.st.com/en/development-tools/stm32cubemx.html>
- Web server technology: Django, Flask, NodeJS, PHP
- Database technology: MariaDB, PostgreSQL, SQLite
- Machine learning framework: TensorFlow, PyTorch

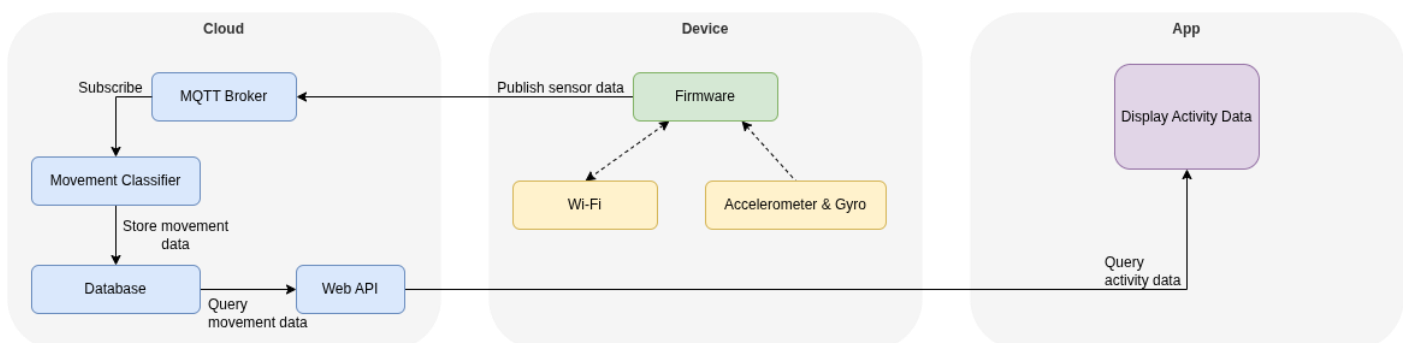
Tasks

The project involves three key tasks:

1. Development of the cloud service and machine learning model
2. Development of the device firmware
3. Development of the mobile app

These tasks should be tackled in this order so that, when it comes to developing the device firmware, you can connect it to your cloud service immediately.

High Level Overview



This figure gives a high-level overview of how your system should fit together. Your embedded device should periodically send sensor data to the cloud over an MQTT connection. In the cloud, a movement classifier should read the sensor data, classify it, and store it in a database. A web API should allow access to the data. Your mobile app should query the data, through the web API, and display it on screen.

Cloud Service

Your cloud service will comprise:

- An MQTT broker for receiving messages from the IoT device
- A machine learning-based classifier to classify sensor data into movement activity.
- A database for storing processed activity data
- A web API server for accessing the stored data

To do this, you should set up and prepare a virtual machine on a cloud service provider, such as Amazon Web Services, Google Cloud, or Microsoft Azure. You can then configure your virtual machine with all the software you need. Often, cloud service providers have a free tier, which lets you experiment. The process for creating a virtual machine is different for each provider. However, once you have the virtual machine, the operating system is the same.

If you are familiar with Docker containers, you could also develop this service locally. For example, you could run a container for the MQTT broker, one for the classifier, one for the database, and one for the web API. You could use Docker Compose to build a service deployment.

The MQTT broker is what your embedded device connects to and transmits sensor data to. The classic MQTT broker is Mosquitto, which is available as a Linux package in all popular Linux distributions.

You could use an SQL database such as MariaDB or PostgreSQL for your data storage, or you could use a NoSQL database such as MongoDB.

You can write your Web API in a range of different technologies:

- Python
- PHP
- Java
- Rust
- Node.js

Ideally, you'd expose API endpoints to allow classified movement data to be returned, and displayed on a web page, or, ultimately, in your mobile app.

Machine Learning Models

To achieve activity recognition, you will leverage the power of machine learning. You may use the following dataset on Kaggle: <https://www.kaggle.com/vmalyi/run-or-walk>, which contains previously collected sensor data from accelerometers and gyroscopes. The documentation explains how the data was collected and what model was used for training.

You are strongly encouraged to read through the accompanying documentation, in case you hope to collect additional data later, but also if you wish to enhance the robustness of your model or diversify the detection

scenarios. Even if you do not want to do so, the original documentation contain a key hint about the measurement unit used to represent the sensor data, which is not recorded explicitly in the dataset.

However, you are not advised to use the same model as proposed by the original author. You may find that the proposed method does not make full use of readings from all axes. Here is one important point: you should do your best to make sure that your model can generalize well across multiple rigorous testing scenarios. To achieve this, you need to make your model learn as much useful information as possible.

In terms of model design, you may consider the following aspects, to ensure your model achieves good performance:

1. **Make use of temporal information**
Clearly, motion detection is a pattern recognition task on time-series multi-dimensional data. In other words, the task is time-sensitive, so you need to ensure that the model can learn temporal correlations from the dataset. For some time-sensitive tasks (even for this one), it might be possible that a time-insensitive model can exhibit reasonable performance. However, be aware that if your model does not learn temporal correlations from the dataset, you may only gain limited insights.
2. **Keep things lightweight**
Activity tracking is not a difficult learning task that requires thousands of parameters and days of training. When designing a model, you should start with the simplest solution and gradually add complexity, rather than going directly for a sophisticated deep learning model. Even though all the computation can be performed in the cloud, you should not use more resources than you need. Different models use different resources by default. Before you start, consider the following questions: Is it better to choose a sophisticated model that should achieve excellent performance but has many parameters and requires a large amount of training data, or to choose a model with fewer parameters that can be trained in one batch? Do you value computational resources or memory resources more? Do you want your model to provide probabilistic interpretations?

Device Firmware

Once you're happy that your cloud service is working, you can develop your device firmware and start to send real data. Much of the firmware will be based on what you've learned previously, but with the addition of using the on-board Wi-Fi interface and sending data using the MQTT protocol. You'll be using the on-board accelerometer and gyroscope (the LSM6DSL), so make sure this software pack has been added in the CubeMX generator.

To get Wi-Fi working on the development board, you'll also need to activate the **SPI3** global interrupt. Go to the "Connectivity" section, click SPI3, navigate to "NVIC Settings", and select the global interrupt checkbox.

Configuration			
Reset Configuration			
✓ Parameter Settings	✓ User Constants	✓ NVIC Settings	✓ DMA Settings
✓ GPIO Settings			
NVIC Interrupt Table	Enabled	Preemption Priority	Sub Priority
SPI3 global interrupt	☒	0	0

Once you have generated the code, there are a few bits of configuration to take care of. First, you'll need to install the appropriate Wi-Fi driver pack, along with the MQTT implementation. In the terminal, run the following commands:

```
cpackget add ARM::CMSIS-Driver
cpackget add AWS::coreMQTT
```

This will download the CMSIS Driver and MQTT packs, but you also need to include them in your solution. Open the component configuration window for the solution, then enable the following:

- CMSIS Driver
 - SPI (API) -> SPI
 - WiFi (API) -> ISM43362
- FreeRTOS
 - coreMQTT
- Network
 - Core
 - Interface -> Wi-Fi
 - Socket -> UDP

Component	Variant	Version	Vendor	Description
> CMSIS (3 of 3)				Cortex Microcontroller Software Interface Components
> CMSIS Driver (5 of 58)				Unified Device Drivers compliant to CMSIS-Driver Specifications
> CMSIS-Compiler (4 of 20)				Compiler Specific Interfaces
> Device (1 of 1)				Startup, System Setup
> File System (0 of 6)	MDK			File Access on various storage devices
▼ FreeRTOS (1 of 1)				
coreMQTT	☒	5.0.2	AWS	Client implementation of the MQTT 5.0 specification for embedded devices.
▼ Network (3 of 19)	MDK			IPv4/IPv6 Networking using Ethernet or Serial protocols
CORE	☒	8.1.0	Keil	IPv4/IPv6 Networking Core for Cortex-M
▼ Interface (1 of 4)				
ETH	☐	8.1.0	Keil	Network Ethernet Interface
PPP	☐	Custom Modem	8.1.0	Keil Network PPP over Serial Interface
SLIP	☐	Custom Modem	8.1.0	Keil Network SLIP Interface
WiFi	☒	8.1.0	Keil	Network WiFi Interface
> Service (0 of 11)				
▼ Socket (1 of 3)				
BSD	☐	8.1.0	Keil	BSD Socket
TCP	☐	8.1.0	Keil	TCP Socket
UDP	☒	8.1.0	Keil	UDP Socket
> USB (0 of 12)	MDK			USB Communication with various device classes

Component	Variant	Version	Vendor	Description
> I2C (API) (0 of 3)				I2C (Inter-Integrated Circuit) Driver API
> MCI (API) (0 of 2)				MCI (Memory Card Interface) Driver API
> NAND (API) (0 of 2)				NAND Flash Driver API
> SAI (API) (0 of 1)				SAI (Serial Audio Interface) Driver API
▼ SPI (API) (1 of 3)				SPI (Serial Peripheral Interface) Driver API
Custom	☐	1.0.0	ARM	Access to #include Driver_SPI.h file and code template for custom implementation
Multi-Slave	☐	1.0.1	Keil	SPI Master to Multi-Slave Driver Wrapper
SPI	☒	3.0.0	Keil	SPI Driver for STM32 devices
> USART (API) (1 of 4)				USART (Universal Synchronous/Asynchronous Receiver/Transmitter) Driver API
> USB Device (API) (0 of 2)				USB (Universal Serial Bus) Device Driver API
> USB Host (API) (0 of 4)				USB (Universal Serial Bus) Host Driver API
> VIO (API) (1 of 3)				Virtual I/O abstraction for simple peripherals (i.e. button, LED)
> vStream (API) (0 of 1)				Virtual Streaming interface Driver API
▼ WiFi (API) (1 of 6)				WiFi (Wireless network) Driver API
Custom	☐	1.0.0	ARM	Access to #include Driver_WiFi.h file
DA16200	☐	1.0.0	Keil	Renesas DA16200 WiFi Driver (UART)
ESP32	☐	1.7.0	Keil	Espressif ESP32 WiFi Driver (UART)
ESP8266	☐	1.8.0	Keil	Espressif ESP8266 WiFi Driver (UART)
ISM43362	☒	1.15.0	Keil	WiFi Inventek ISM43362/ISM43340 Driver (SPI)
WizFi360	☐	1.8.0	Keil	WIZnet WizFi360 WiFi Driver (UART)

This will enable the MQTT component and the Wi-Fi driver. There are some additional manual configuration steps that you need to complete the board setup.

First, edit your solution.yml file, and add the following to the end:

```
define:
  - MQTT_DO_NOT_USE_CUSTOM_CONFIG
```

It should come after “compiler: AC6” and be indented at the same level. This tells the MQTT component to use its default configuration. Next, you need to modify the WiFi_ISM43362_HW.c file to connect the Wi-Fi driver to the appropriate GPIO pins. You’ll find this file under RTE/CMSIS_Driver. Use the provided reference file to populate it. If you’re using a different embedded device, you’ll need to modify the GPIO pin definitions accordingly.

Once this is done, you’re ready to connect your board to Wi-Fi. Take inspiration from the IoT Sensors Lab to build your firmware. Your firmware should roughly follow this sequence:

- Initialise sensors
- Initialise Wi-Fi
- Connect to the MQTT broker
- Take a sensor reading every second and send an MQTT message

Here is some sample code for connecting to Wi-Fi. This code is missing error checking, so make sure you adapt it as necessary, but it should be good enough to get you started:

```
Driver_WiFi0.Initialize(NULL);
Driver_WiFi0.PowerControl(ARM_POWER_FULL);
```

```
ARM_WIFI_CONFIG_t wifiConfig = {0};  
wifiConfig.ssid = "SSID";           // Your network SSID.  
wifiConfig.pass = "PASSWORD";       // Your network password.  
wifiConfig.security = ARM_WIFI_SECURITY_WPA2; // Your network security mode.  
wifiConfig.ch = 0;  
wifiConfig.wps_method = ARM_WIFI_WPS_METHOD_NONE;  
  
Driver_WiFi0.Activate(0, &wifiConfig);
```

You'll find the documentation for using the coreMQTT library here: <https://docs.aws.amazon.com/freertos/latest/userguide/coremqtt.html>

Remember, you'll have to publish your MQTT message in the format expected by your cloud service.

Mobile App

You should use an Android smartphone to develop a mobile app to display relevant information retrieved from the cloud.

Again, Android Studio is the most accessible tool for developing an app, and this lab is deliberately opened to let you explore. Indeed, feel free to choose Java or Kotlin as the programming language, depending on what you'd like to learn. Alternatively, if you are more familiar with JavaScript, there are a number of JavaScript mobile frameworks that you can also choose, such as Cordova and NativeScript.

If you are using Android Studio, the new part of this lab will be HTTP communication. The recommendation is to use Volley, an HTTP library developed by Google, to communicate with the cloud service. More information about Volley can be found at <https://developer.android.com/training/volley>.

Android strictly discourages I/O-intensive operations, such as networking, from running in the UI thread (that is the main thread). Normally, HTTP requests are wrapped in an AsyncTask class and a callback function needs to be defined, which can significantly affect productivity. Volley provides convenient APIs for various HTTP requests that can substantially reduce the amount of code.

There are no specific requirements for the design of the mobile app, but in general it should have the following functionalities:

1. Display the real-time predictions retrieved from the cloud
2. Display a history of activity records stored in the database in the cloud
3. Ensure the user can interact with the app smoothly and that the code is bug-free.

Attractive UIs and animations are also highly encouraged, and remember to pay attention to the software engineering as well as by decomposing functionality into additional classes and methods.